Tutorial React Native: Componentes, Flexbox e Navegação

Oct 22, 2026 · @Daniel Corrêa Da Silva

Etapa 1: Criando o projeto com o template blank

O projeto é criado com o Expo, que já traz o ambiente pronto para rodar no celular (app Expo Go), no emulador Android/iOS ou no navegador. O template **blank** gera o projeto mínimo: um único arquivo `App.js`, sem navegação nem telas de exemplo, ideal para aprender do zero.

Conceitos:

- **Expo:** plataforma que simplifica o desenvolvimento React Native (build, acesso a câmera, GPS etc.) sem configurar Android Studio/Xcode no início.
- **Expo Go:** app instalado no celular que lê o QR Code do terminal e executa o projeto.
- **Metro Bundler:** servidor que empacota o JavaScript e envia ao dispositivo; atualiza a tela a cada salvamento (Fast Refresh).

Comandos Bash:

```
# Cria o projeto usando o template blank (JavaScript)
npx create-expo-app@latest meu-app --template blank

# Alternativa com TypeScript
# npx create-expo-app@latest meu-app --template blank-typescript

# Entra na pasta do projeto
cd meu-app

# Inicia o servidor de desenvolvimento (Metro)
npx expo start
```

No terminal aparecerá um QR Code. Atalhos úteis: `a` abre no emulador Android, `i` no simulador iOS (somente macOS), `w` no navegador e `r` recarrega o app.

> **Dica de laboratório:** se o celular e o computador não estiverem na mesma rede (ou a rede bloquear a conexão), use `npx expo start --tunnel`.

Estrutura gerada:

```
meu-app/
├── assets/          # ícones e splash screen
├── App.js           # componente raiz: ponto de partida do app
├── app.json         # configurações do Expo (nome, ícone, orientação)
├── index.js         # registra o App.js como componente principal
└── package.json     # dependências e scripts
```

Código inicial (`App.js`):

```
// StatusBar do Expo controla a barra de status (hora, bateria) do celular
import { StatusBar } from 'expo-status-bar';
// Componentes básicos do React Native
import { StyleSheet, Text, View } from 'react-native';

// Todo app React Native é uma função que retorna JSX
export default function App() {
  return (
    // View = contêiner, equivalente a uma <div> no React web
    <View style={styles.container}>
      {/* Text = único componente que pode exibir texto */}
      <Text>Open up App.js to start working on your app!</Text>
      <StatusBar style="auto" />
    </View>
  );
}
```

```
// Estilos definidos fora do componente (veja a Etapa 3)
const styles = StyleSheet.create({
  container: {
    flex: 1,                    // ocupa toda a tela
    backgroundColor: '#fff',
    alignItems: 'center',       // centraliza na horizontal
    justifyContent: 'center',   // centraliza na vertical
  },
});
```

> **Diferença para o React web:** não existem `<div>`, `<p>` ou `<span>`. Usamos componentes nativos (`View`, `Text`, `Image`...) que viram componentes reais do Android e do iOS.

Etapa 2: Componentes View, ScrollView e Text

`View` organiza o layout, `ScrollView` permite rolar conteúdo maior que a tela e `Text` exibe texto. Esses três componentes formam a base de praticamente todas as telas.

2.1 View

Conceito: `View` é o contêiner fundamental, equivalente à `<div>`. Agrupa outros componentes e recebe estilos (cor, borda, espaçamento, Flexbox). Toda `View` já é um contêiner Flexbox com `flexDirection: 'column'` por padrão.

Exemplo (`App.js`):

```
import { View } from 'react-native';

export default function App() {
  return (
    // View externa: ocupa a tela toda e empilha as filhas na vertical
    <View style={{ flex: 1, padding: 20, backgroundColor: '#f0f0f0' }}>
      {/* Três caixas coloridas: cada View é um retângulo */}
      <View style={{ height: 80, backgroundColor: '#e74c3c' }} />
      <View style={{ height: 80, backgroundColor: '#2ecc71' }} />
      <View style={{ height: 80, backgroundColor: '#3498db' }} />
    </View>
  );
}
```

> **Regra importante:** texto nunca pode ficar solto dentro de uma `View`. Escrever `<View>Olá</View>` gera o erro *"Text strings must be rendered within a <Text> component"*.

2.2 Text

Conceito: `Text` é o único componente que exibe texto. Ele aceita aninhamento (um `Text` dentro de outro herda o estilo do pai) e a propriedade `numberOfLines`, que corta textos longos com reticências.

Exemplo:

```
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View style={{ flex: 1, padding: 40 }}>
      {/* Texto com estilo próprio */}
      <Text style={{ fontSize: 24, fontWeight: 'bold' }}>Título</Text>

      {/* Text aninhado: a palavra "negrito" herda o tamanho do pai */}
      <Text style={{ fontSize: 16 }}>
        Este texto possui uma palavra em{' '}
        <Text style={{ fontWeight: 'bold', color: 'red' }}>negrito</Text>.
      </Text>

      {/* numberOfLines limita a 2 linhas e adiciona "..." ao final */}
      <Text numberOfLines={2} style={{ marginTop: 10 }}>
        Um texto muito longo que não cabe em duas linhas será cortado
        automaticamente com reticências, útil em cards e listas de produtos.
      </Text>
```

```
      {/* onPress: Text também pode ser clicável (ex.: link "Esqueci a senha") */}
      <Text onPress={() => alert('Clicou!')} style={{ color: 'blue' }}>
        Clique aqui
      </Text>
    </View>
  );
}
```

> **Diferença para o web:** no React Native a herança de estilo só acontece de `Text` para `Text`. Uma `View` com `color: 'red'` não muda a cor dos textos dentro dela.

2.3 ScrollView

Conceito: a `View` não rola: se o conteúdo for maior que a tela, ele é cortado. A `ScrollView` adiciona rolagem. Ela renderiza **todos** os filhos de uma vez, por isso é indicada para conteúdo pequeno ou médio (formulários, telas de detalhes). Para listas grandes (centenas de itens), use `FlatList`, que renderiza só o que está visível.

Propriedades principais:

Propriedade	O que faz
`horizontal`	Rola na horizontal em vez da vertical
`contentContainerStyle`	Estilo do conteúdo interno (use para `padding`, `alignItems`, `gap`)
`showsVerticalScrollIndicator`	Mostra ou esconde a barra de rolagem
`keyboardShouldPersistTaps`	Com `'handled'`, permite tocar em botões com o teclado aberto

Exemplo:

```
import { ScrollView, View, Text, StyleSheet } from 'react-native';

// Cria um array com 20 números para simular conteúdo longo
const itens = Array.from({ length: 20 }, (_, i) => i + 1);

export default function App() {
  return (
    // style: estilo da "janela" de rolagem (ocupa a tela)
    // contentContainerStyle: estilo do conteúdo que rola
    <ScrollView
      style={styles.tela}
      contentContainerStyle={styles.conteudo}
      showsVerticalScrollIndicator={false}
    >
      {/* Lista horizontal de categorias (ScrollView dentro de ScrollView) */}
      <ScrollView horizontal showsHorizontalScrollIndicator={false}>
        {['Todos', 'Bebidas', 'Lanches', 'Doces', 'Promoções'].map((cat) => (
          // key: identificador único exigido pelo React em listas
          <Text key={cat} style={styles.chip}>{cat}</Text>
        ))}
      </ScrollView>

      {/* Lista vertical de cartões */}
      {itens.map((n) => (
        <View key={n} style={styles.card}>
          <Text>Item {n}</Text>
        </View>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  tela: { flex: 1, backgroundColor: '#fff' },
  conteudo: { padding: 16, paddingTop: 60, gap: 10 },
  chip: {
    paddingHorizontal: 14,
    paddingVertical: 6,
```

```
    marginRight: 8,
    borderRadius: 16,
    backgroundColor: '#eee',
  },
  card: { padding: 20, borderRadius: 8, backgroundColor: '#dfe6e9' },
});
```

> **Erro comum:** colocar `alignItems` ou `justifyContent` em `style` da ScrollView gera aviso. Esses estilos vão em `contentContainerStyle`.

Etapa 3: StyleSheet.create, Height e Width

3.1 StyleSheet.create

Conceito: no React Native não existe CSS. Os estilos são objetos JavaScript com propriedades em **camelCase** (`backgroundColor` em vez de `background-color`) e valores numéricos **sem unidade** (16 significa 16 pontos independentes de densidade, ou *dp*). `StyleSheet.create` agrupa esses objetos fora do componente.

Vantagens de usar `StyleSheet.create` em vez de estilo inline:

- **Organização:** separa a aparência da estrutura, como um arquivo CSS.
- **Validação:** em desenvolvimento, alerta sobre propriedades inexistentes.
- **Desempenho:** o objeto é criado uma única vez, e não a cada renderização.

Exemplo:

```
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Meu Card</Text>

      {/* Array de estilos: os da direita sobrescrevem os da esquerda */}
      <View style={[styles.card, styles.cardDestaque]}>
        <Text style={styles.texto}>Card com destaque</Text>
      </View>

      {/* Estilo condicional: aplica styles.ativo apenas se ativo for true */}
      <View style={[styles.card, true && styles.ativo]}>
        <Text style={styles.texto}>Card ativo</Text>
      </View>
    </View>
  );
}

// Definido fora do componente: criado uma única vez
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 24,
    paddingTop: 60,
    backgroundColor: '#fafafa',
  },
  titulo: {
    fontSize: 22,          // sem 'px'
    fontWeight: 'bold',    // valores de texto entre aspas
    marginBottom: 16,
  },
  card: {
    padding: 16,
    marginBottom: 12,
    borderRadius: 8,       // cantos arredondados
    borderWidth: 1,
    borderColor: '#ddd',
    backgroundColor: '#fff',
  },
  cardDestaque: {
```

```
    borderColor: '#6c5ce7', // sobrescreve borderColor de card
    borderWidth: 2,
  },
  ativo: { backgroundColor: '#dff9fb' },
  texto: { fontSize: 16, color: '#333' },
});
```

3.2 Height e Width

Conceito: a largura (`width`) e a altura (`height`) de um componente podem ser definidas de três formas:

Forma	Exemplo	Quando usar
Fixa (dp)	`width: 100`	Ícones, avatares, botões de tamanho conhecido
Percentual	`width: '50%'`	Proporção relativa ao **pai**
Flexível	`flex: 1`	Ocupar o espaço restante (detalhado na Etapa 4)

Também existem limites: `minWidth` , `maxWidth` , `minHeight` e `maxHeight` . Para descobrir o tamanho da tela, use o hook `useWindowDimensions` , que se atualiza quando o celular gira.

Exemplo:

```
import { View, Text, StyleSheet, useWindowDimensions } from 'react-native';

export default function App() {
  // Retorna largura e altura da janela; recalcula ao girar a tela
  const { width, height } = useWindowDimensions();

  return (
    <View style={styles.container}>
      <Text>Tela: {Math.round(width)} x {Math.round(height)} dp</Text>

      {/* 1) Dimensão fixa: sempre 100 x 100 */}
      <View style={[styles.caixa, { width: 100, height: 100 }]}>
        <Text>100x100</Text>
      </View>

      {/* 2) Percentual: metade da largura do pai */}
      <View style={[styles.caixa, { width: '50%', height: 60 }]}>
        <Text>50%</Text>
      </View>

      {/* 3) Calculada a partir da tela: um quadrado com 1/3 da largura */}
      <View style={[styles.caixa, { width: width / 3, height: width / 3 }]}>
        <Text>1/3 da tela</Text>
      </View>

      {/* 4) Limite máximo: em tablets não passa de 400 */}
      <View style={[styles.caixa, { width: '100%', maxWidth: 400, height: 50 }]}>
        <Text>maxWidth 400</Text>
      </View>

      {/* 5) flex: 1 ocupa toda a altura que sobrou */}
      <View style={[styles.caixa, { flex: 1 }]}>
        <Text>flex: 1 (resto da tela)</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, paddingTop: 60, gap: 10 },
  caixa: {
    backgroundColor: '#81ecec',
    alignItems: 'center',
    justifyContent: 'center',
```

```
  },
});
```

> **Atenção:** percentuais dependem do pai ter tamanho definido. Um `height: '50%'` dentro de uma `View` sem altura (nem `flex`) não tem referência e não funciona como esperado.

Etapa 4: Flexbox — propriedades e exemplos

Flexbox é o sistema de layout do React Native: o **pai** (contêiner) decide a direção e o alinhamento, e os **filhos** (itens) decidem quanto espaço ocupam. Ele funciona como no CSS, com três diferenças de padrão:

Propriedade	Padrão no CSS web	Padrão no React Native
`flexDirection`	`row`	`column`
`alignContent`	`normal` / `stretch`	`flex-start`
`flexShrink`	`1`	`0`

Conceito-chave — os dois eixos:

- **Eixo principal (main axis):** segue o `flexDirection`. É controlado por `justifyContent`.
- **Eixo transversal (cross axis):** perpendicular ao principal. É controlado por `alignItems`, `alignSelf` e `alignContent`.

Com `flexDirection: 'column'` o eixo principal é vertical; com `'row'`, horizontal. Por isso `justifyContent: 'center'` centraliza na vertical em uma coluna e na horizontal em uma linha.

Componente auxiliar usado nos exemplos: para não repetir código, todos os exemplos usam esta caixa colorida. Coloque-a no topo do `App.js`.

```
import { View, Text, StyleSheet } from 'react-native';

// Caixa colorida reutilizável: recebe uma cor, um rótulo e estilos extras
function Caixa({ cor, rotulo, style }) {
  return (
    <View style={[{ width: 60, height: 60, backgroundColor: cor,
      alignItems: 'center', justifyContent: 'center' }, style]}>
      <Text style={{ color: '#fff', fontWeight: 'bold' }}>{rotulo}</Text>
    </View>
  );
}
```

4.1 flexDirection (direction)

Conceito: define o eixo principal, ou seja, em que sentido os filhos são empilhados.

Valor	Resultado
`column` (padrão)	De cima para baixo
`column-reverse`	De baixo para cima
`row`	Da esquerda para a direita
`row-reverse`	Da direita para a esquerda

```
export default function App() {
  return (
    // Troque 'row' por 'column', 'row-reverse' ou 'column-reverse' e observe
    <View style={{ flex: 1, paddingTop: 60, flexDirection: 'row' }}>
      <Caixa cor="#e74c3c" rotulo="1" />
      <Caixa cor="#2ecc71" rotulo="2" />
      <Caixa cor="#3498db" rotulo="3" />
    </View>
  );
}
```

> **Direção do texto (`direction`):** existe também a propriedade `direction` (`'ltr'` ou `'rtl'`), que inverte o sentido de leitura para idiomas como árabe e hebraico. Em `rtl`, `flexDirection: 'row'` passa a começar pela direita.

4.2 justifyContent

Conceito: distribui os filhos ao longo do **eixo principal**.

Valor	Resultado
`flex-start` (padrão)	Agrupados no início
`flex-end`	Agrupados no fim
`center`	Agrupados no centro
`space-between`	Primeiro no início, último no fim, espaço igual entre eles
`space-around`	Espaço igual ao redor de cada item (as bordas recebem metade)
`space-evenly`	Espaços exatamente iguais, inclusive nas bordas

```
export default function App() {
  return (
    // Coluna (padrão): justifyContent atua na VERTICAL
    <View style={{ flex: 1, paddingVertical: 60, justifyContent: 'space-between' }}>
      <Caixa cor="#e74c3c" rotulo="1" />
      <Caixa cor="#2ecc71" rotulo="2" />
      <Caixa cor="#3498db" rotulo="3" />
    </View>
  );
}
```

Uso real: um cabeçalho com título à esquerda e ícone à direita usa `flexDirection: 'row'` + `justifyContent: 'space-between'`.

4.3 alignItems

Conceito: alinha os filhos no **eixo transversal**. É definida no pai e vale para todos os filhos.

Valor	Resultado
`stretch` (padrão)	Estica o filho para preencher o eixo transversal (só se ele não tiver tamanho fixo nesse eixo)
`flex-start`	Alinhados no início
`flex-end`	Alinhados no fim
`center`	Centralizados
`baseline`	Alinhados pela linha de base do texto

```
export default function App() {
  return (
    // Coluna: alignItems atua na HORIZONTAL
    <View style={{ flex: 1, paddingTop: 60, alignItems: 'center' }}>
      <Caixa cor="#e74c3c" rotulo="1" />
      <Caixa cor="#2ecc71" rotulo="2" />

      {/* Sem width, com stretch esta caixa ocuparia a largura toda */}
      <Caixa cor="#3498db" rotulo="3" style={{ width: 120 }} />
    </View>
  );
}

// Centralizar algo no meio exato da tela = os dois eixos:
// { flex: 1, justifyContent: 'center', alignItems: 'center' }
```

4.4 alignSelf

Conceito: definida no **filho**, sobrescreve o `alignItems` do pai apenas para aquele item. Aceita os mesmos valores, mais `auto` (padrão: herda do pai).

```
export default function App() {
  return (
    // Pai alinha todos no início (esquerda)...
    <View style={{ flex: 1, paddingTop: 60, alignItems: 'flex-start' }}>
      <Caixa cor="#e74c3c" rotulo="1" />
      {/* ...mas este filho vai para o centro */}
      <Caixa cor="#2ecc71" rotulo="2" style={{ alignSelf: 'center' }} />
      {/* ...e este para o fim (direita) */}
      <Caixa cor="#3498db" rotulo="3" style={{ alignSelf: 'flex-end' }} />
      {/* stretch: estica (precisa remover o width fixo) */}
      <Caixa cor="#9b59b6" rotulo="4" style={{ alignSelf: 'stretch', width: undefined }} />
    </View>
  );
}
```

Uso real: em um chat, as mensagens enviadas ficam com `alignSelf: 'flex-end'` e as recebidas com `alignSelf: 'flex-start'`.

4.5 flexWrap

Conceito: por padrão (`nowrap`), todos os filhos ficam em uma única linha, mesmo que ultrapassem a tela. Com `wrap` , os itens que não cabem quebram para a próxima linha. `wrap-reverse` quebra no sentido contrário.

```
// Gera 10 caixas
const cores = ['#e74c3c', '#2ecc71', '#3498db', '#9b59b6', '#f39c12'];

export default function App() {
  return (
    <View style={{ flex: 1, paddingTop: 60, flexDirection: 'row', flexWrap: 'wrap' }}>
      {Array.from({ length: 10 }, (_, i) => (
        // Cada caixa tem 80 de largura: em ~400dp cabem 5 por linha
        <Caixa key={i} cor={cores[i % 5]} rotulo={i + 1} style={{ width: 80 }} />
      ))}
    </View>
  );
}
```

Uso real: grades de produtos, galerias de fotos e etiquetas (tags).

4.6 alignContent

Conceito: distribui as **linhas** no eixo transversal quando há quebra de linha. Só tem efeito com `flexWrap: 'wrap'` e mais de uma linha. Pense assim: `alignItems` alinha itens dentro de uma linha; `alignContent` alinha o conjunto de linhas.

Valores: `flex-start` (padrão no RN), `flex-end` , `center` , `stretch` , `space-between` , `space-around` e `space-evenly` .

```
export default function App() {
  return (
    <View
      style={{
        height: 400,               // altura sobrando para ver a distribuição
        marginTop: 60,
        backgroundColor: '#eee',
        flexDirection: 'row',
        flexWrap: 'wrap',          // obrigatório para alignContent funcionar
        alignContent: 'space-between', // teste: center, flex-end, space-around
      }}
    >
      {Array.from({ length: 10 }, (_, i) => (
        <Caixa key={i} cor={cores[i % 5]} rotulo={i + 1} style={{ width: 80 }} />
      ))}
    </View>
  );
}
```

4.7 flexBasis, flexGrow e flexShrink

Conceito: essas três propriedades ficam no **filho** e controlam o tamanho dele no eixo principal.

Propriedade	Pergunta que responde	Padrão
`flexBasis`	Qual o tamanho **inicial** do item antes de dividir o espaço?	`auto` (usa width/height)
`flexGrow`	Se **sobrar** espaço, quanto deste item cresce?	`0` (não cresce)
`flexShrink`	Se **faltar** espaço, quanto deste item encolhe?	`0` no RN (não encolhe)

O atalho `flex: N` (número positivo) equivale a `flexGrow: N`, `flexShrink: 1` e `flexBasis: 0`. Por isso `flex: 1` significa "ocupe o espaço restante".

Exemplo de flexGrow (proporções):

```
export default function App() {
  return (
    <View style={{ flex: 1, paddingTop: 60 }}>
      {/* Espaço restante dividido em 1 + 2 + 1 = 4 partes */}
      <View style={{ flexGrow: 1, backgroundColor: '#e74c3c' }} />  {/* 1/4 */}
      <View style={{ flexGrow: 2, backgroundColor: '#2ecc71' }} />  {/* 2/4 */}
      <View style={{ flexGrow: 1, backgroundColor: '#3498db' }} />  {/* 1/4 */}
    </View>
  );
}
```

Exemplo de flexBasis + flexGrow (linha):

```
export default function App() {
  return (
    <View style={{ flexDirection: 'row', marginTop: 60, height: 80 }}>
      {/* Começa com 100 e NÃO cresce */}
      <View style={{ flexBasis: 100, backgroundColor: '#e74c3c' }} />
      {/* Começa com 50 e fica com TODO o espaço que sobrar */}
      <View style={{ flexBasis: 50, flexGrow: 1, backgroundColor: '#2ecc71' }} />
    </View>
  );
}
```

Exemplo de flexShrink (falta de espaço):

```
export default function App() {
  return (
    // Os filhos somam 600 de largura, mais que a tela (~400)
    <View style={{ flexDirection: 'row', marginTop: 60, height: 80 }}>
      {/* flexShrink 0: mantém os 200 de qualquer forma */}
      <View style={{ width: 200, flexShrink: 0, backgroundColor: '#e74c3c' }} />
      {/* flexShrink 1: encolhem igualmente para caber na tela */}
      <View style={{ width: 200, flexShrink: 1, backgroundColor: '#2ecc71' }} />
      <View style={{ width: 200, flexShrink: 1, backgroundColor: '#3498db' }} />
    </View>
  );
}
```

Uso real do flexShrink: em uma linha com ícone + texto longo, coloque `flexShrink: 1` no `Text` para que ele quebre a linha em vez de empurrar o conteúdo para fora da tela.

4.8 rowGap, columnGap e gap

Conceito: definem o espaço **entre** os filhos, sem precisar de `margin` em cada um. Ficam no pai.

Propriedade	Espaço entre
`rowGap`	Linhas (espaço vertical)
`columnGap`	Colunas (espaço horizontal)
`gap`	Atalho para os dois

```
export default function App() {
  return (
    <View
      style={{
        marginTop: 60,
        padding: 10,
        flexDirection: 'row',
        flexWrap: 'wrap',
        rowGap: 20,     // 20 entre uma linha e outra
        columnGap: 8,   // 8 entre caixas da mesma linha
        // gap: 10,     // alternativa: mesmo valor nos dois sentidos
      }}
    >
      {Array.from({ length: 9 }, (_, i) => (
        <Caixa key={i} cor={cores[i % 5]} rotulo={i + 1} style={{ width: 100 }} />
      ))}
    </View>
  );
}
```

> **Vantagem sobre margin:** o `gap` não adiciona espaço antes do primeiro nem depois do último item, então não é preciso tratar as bordas.

4.9 position

Conceito: define como o item é posicionado.

- **`relative` (padrão):** o item participa do Flexbox normalmente. `top`, `left`, `right` e `bottom` apenas o deslocam a partir da posição original.
- **`absolute`:** o item **sai do fluxo** do Flexbox (não ocupa espaço nem empurra os irmãos) e é posicionado em relação ao pai, usando `top`, `left`, `right` e `bottom`.

Use `zIndex` para controlar qual elemento fica por cima.

```
export default function App() {
  return (
    <View style={{ flex: 1, paddingTop: 60, alignItems: 'center' }}>
      {/* RELATIVE: deslocada 20 para a direita da posição normal */}
      <Caixa cor="#e74c3c" rotulo="rel" style={{ position: 'relative', left: 20 }} />

      {/* Pai de referência para o absolute */}
      <View style={{ width: 120, height: 120, marginTop: 30, backgroundColor: '#3498db' }}>
        {/* ABSOLUTE: "badge" no canto superior direito do pai */}
        <View
          style={{
            position: 'absolute',
            top: -10,
            right: -10,
            width: 30,
            height: 30,
            borderRadius: 15,
            backgroundColor: '#e74c3c',
            alignItems: 'center',
            justifyContent: 'center',
            zIndex: 1,
          }}
        >
          <Text style={{ color: '#fff' }}>3</Text>
        </View>
      </View>

      {/* Botão flutuante (FAB) fixo no canto inferior direito da tela */}
      <View
        style={{
          position: 'absolute',
          bottom: 30,
          right: 30,
          width: 56,
```

```
          height: 56,
          borderRadius: 28,
          backgroundColor: '#6c5ce7',
          alignItems: 'center',
          justifyContent: 'center',
        }}
      >
        <Text style={{ color: '#fff', fontSize: 28 }}>+</Text>
      </View>
    </View>
  );
}
```

Usos reais do absolute: badges de notificação, botão flutuante, texto sobre imagem e sobreposições (overlays). Dica: `...StyleSheet.absoluteFillObject` cobre o pai inteiro (top, left, right e bottom iguais a 0).

Resumo: onde cada propriedade fica

No pai (contêiner)	No filho (item)
`flexDirection`, `justifyContent`, `alignItems`	`alignSelf`
`flexWrap`, `alignContent`	`flex`, `flexGrow`, `flexShrink`, `flexBasis`
`gap`, `rowGap`, `columnGap`	`position`, `top`, `left`, `right`, `bottom`, `zIndex`

Etapa 5: Estudo de caso — layout das telas de Login, Registro e Principal

As três telas são montadas apenas com Flexbox: cada tela é dividida em blocos (cabeçalho, formulário, rodapé), e cada bloco usa as propriedades da Etapa 4. Nesta etapa os botões de navegação apenas chamam `navigation.navigate(...)`; a ligação real entre as telas é feita na Etapa 6.

Dois componentes novos usados no formulário:

- **`TextInput`:** campo de entrada de texto. Propriedades úteis: `placeholder`, `value`, `onChangeText`, `secureTextEntry` (senha), `keyboardType="email-address"` e `autoCapitalize="none"`.
- **`TouchableOpacity`:** área clicável que fica semitransparente ao toque. Usada para criar botões personalizados (o `Button` nativo quase não aceita estilo).

Estrutura de pastas:

```
meu-app/
├── App.js
└── src/
    ├── screens/
    │   ├── LoginScreen.js
    │   ├── RegisterScreen.js
    │   └── HomeScreen.js
    └── styles/
        └── theme.js      # cores compartilhadas
```

5.1 Tema compartilhado

Código Fonte (`src/styles/theme.js`):

```
// Centraliza as cores: trocar aqui muda o app inteiro
export const cores = {
  primaria: '#6c5ce7',
  fundo: '#f5f6fa',
  texto: '#2d3436',
  cinza: '#b2bec3',
  branco: '#ffffff',
};
```

5.2 Tela de Login

Análise do layout: a tela é uma coluna (`flex: 1`) com conteúdo centralizado na vertical (`justifyContent: 'center'`). Ela tem três blocos: cabeçalho (logo + título), formulário (inputs + botão) e rodapé (link para registro em uma linha, `flexDirection: 'row'`).

```
┌──────────────────────────────┐
│          [ LOGO ]            │  ← cabeçalho: alignItems center
│       Bem-vindo!             │
│                              │
│  [ E-mail              ]     │  ← formulário: gap entre campos
│  [ Senha               ]     │
│         Esqueci a senha →    │  ← alignSelf: flex-end
│  [      ENTRAR         ]     │
│                              │
│ Não tem conta? Cadastre-se   │  ← rodapé: flexDirection row
└──────────────────────────────┘
```

Código Fonte (`src/screens/LoginScreen.js`):

```
import { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity, StyleSheet,
  KeyboardAvoidingView, Platform,
} from 'react-native';
import { cores } from '../styles/theme';

// navigation é recebido automaticamente quando a tela é registrada no navegador (Etapa 6)
export default function LoginScreen({ navigation }) {
  // Estados dos campos do formulário
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  function handleLogin() {
    // Aqui entrará a chamada à API NestJS (POST /auth/login)
    // replace: troca a tela de login pela principal (não dá para "voltar" ao login)
    navigation.replace('Principal');
  }

  return (
    // KeyboardAvoidingView: empurra o conteúdo para cima quando o teclado abre (iOS)
    <KeyboardAvoidingView
      style={styles.container}
      behavior={Platform.OS === 'ios' ? 'padding' : undefined}
    >
      {/* BLOCO 1: cabeçalho */}
      <View style={styles.cabecalho}>
        {/* Logo feita com View: um círculo (borderRadius = metade do tamanho) */}
        <View style={styles.logo}>
          <Text style={styles.logoTexto}>A</Text>
        </View>
        <Text style={styles.titulo}>Bem-vindo!</Text>
        <Text style={styles.subtitulo}>Faça login para continuar</Text>
      </View>

      {/* BLOCO 2: formulário */}
      <View style={styles.formulario}>
        <TextInput
          style={styles.input}
          placeholder="E-mail"
          keyboardType="email-address" // teclado com @
          autoCapitalize="none"        // não coloca maiúscula automática
          value={email}
          onChangeText={setEmail}
        />
        <TextInput
          style={styles.input}
          placeholder="Senha"
          secureTextEntry              // esconde os caracteres
          value={senha}
          onChangeText={setSenha}
```

```
        />

        {/* alignSelf: só este item vai para a direita */}
        <Text style={styles.esqueci}>Esqueci a senha</Text>

        <TouchableOpacity style={styles.botao} onPress={handleLogin}>
          <Text style={styles.botaoTexto}>ENTRAR</Text>
        </TouchableOpacity>
      </View>

      {/* BLOCO 3: rodapé em linha */}
      <View style={styles.rodape}>
        <Text>Não tem conta? </Text>
        <Text style={styles.link} onPress={() => navigation.navigate('Registro')}>
          Cadastre-se
        </Text>
      </View>
    </KeyboardAvoidingView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,                    // ocupa a tela inteira
    justifyContent: 'center',   // blocos no centro vertical
    padding: 24,
    gap: 32,                    // espaço entre os três blocos
    backgroundColor: cores.fundo,
  },
  cabecalho: {
    alignItems: 'center',       // centraliza logo e textos na horizontal
    gap: 8,
  },
  logo: {
    width: 80,
    height: 80,
    borderRadius: 40,
    backgroundColor: cores.primaria,
    justifyContent: 'center',   // centraliza a letra nos dois eixos
    alignItems: 'center',
  },
  logoTexto: { color: cores.branco, fontSize: 36, fontWeight: 'bold' },
  titulo: { fontSize: 26, fontWeight: 'bold', color: cores.texto },
  subtitulo: { color: cores.cinza },
  formulario: {
    gap: 12,                    // sem alignItems: os inputs esticam (stretch)
  },
  input: {
    height: 50,
    paddingHorizontal: 16,
    borderRadius: 10,
    borderWidth: 1,
    borderColor: '#dfe6e9',
    backgroundColor: cores.branco,
  },
  esqueci: {
    alignSelf: 'flex-end',      // sobrescreve o stretch: vai para a direita
    color: cores.primaria,
  },
  botao: {
    height: 50,
    borderRadius: 10,
    backgroundColor: cores.primaria,
    justifyContent: 'center',
    alignItems: 'center',
  },
  botaoTexto: { color: cores.branco, fontWeight: 'bold', fontSize: 16 },
  rodape: {
    flexDirection: 'row',       // textos lado a lado
    justifyContent: 'center',   // centralizados na horizontal
```

```
  },
  link: { color: cores.primaria, fontWeight: 'bold' },
});
```

5.3 Tela de Registro

Análise do layout: o formulário é maior (quatro campos), então usamos `ScrollView` para que nada seja cortado em celulares pequenos ou com o teclado aberto. Os campos "Senha" e "Confirmar" ficam lado a lado com `flexDirection: 'row'` e `flex: 1` em cada um, dividindo a largura igualmente.

Código Fonte (`src/screens/RegisterScreen.js`):

```
import { useState } from 'react';
import { ScrollView, View, Text, TextInput, TouchableOpacity, StyleSheet } from 'react-native';
import { cores } from '../styles/theme';

export default function RegisterScreen({ navigation }) {
  // Um único estado com todos os campos do formulário
  const [form, setForm] = useState({ nome: '', email: '', senha: '', confirmar: '' });

  // Atualiza apenas o campo informado, mantendo os demais (spread)
  function alterar(campo, valor) {
    setForm({ ...form, [campo]: valor });
  }

  function handleRegistrar() {
    if (form.senha !== form.confirmar) {
      alert('As senhas não conferem');
      return;
    }
    // Aqui entrará a chamada à API NestJS (POST /users)
    alert('Conta criada!');
    navigation.goBack(); // volta para a tela anterior (Login)
  }

  return (
    <ScrollView
      style={{ backgroundColor: cores.fundo }}
      contentContainerStyle={styles.conteudo}   // padding e gap vão aqui
      keyboardShouldPersistTaps="handled"         // botão funciona com teclado aberto
    >
      <Text style={styles.titulo}>Criar conta</Text>
      <Text style={styles.subtitulo}>Preencha seus dados</Text>

      <TextInput style={styles.input} placeholder="Nome completo"
        value={form.nome} onChangeText={(v) => alterar('nome', v)} />

      <TextInput style={styles.input} placeholder="E-mail"
        keyboardType="email-address" autoCapitalize="none"
        value={form.email} onChangeText={(v) => alterar('email', v)} />

      {/* Linha com dois campos lado a lado */}
      <View style={styles.linha}>
        <TextInput style={[styles.input, styles.metade]} placeholder="Senha"
          secureTextEntry value={form.senha} onChangeText={(v) => alterar('senha', v)} />
        <TextInput style={[styles.input, styles.metade]} placeholder="Confirmar"
          secureTextEntry value={form.confirmar} onChangeText={(v) => alterar('confirmar', v)} />
      </View>

      <TouchableOpacity style={styles.botao} onPress={handleRegistrar}>
        <Text style={styles.botaoTexto}>CADASTRAR</Text>
      </TouchableOpacity>

      <View style={styles.rodape}>
        <Text>Já tem conta? </Text>
        <Text style={styles.link} onPress={() => navigation.goBack()}>Entrar</Text>
      </View>
    </ScrollView>
  );
```

```
}

const styles = StyleSheet.create({
  conteudo: {
    flexGrow: 1,               // permite centralizar quando o conteúdo é menor que a tela
    justifyContent: 'center',
    padding: 24,
    gap: 12,
  },
  titulo: { fontSize: 26, fontWeight: 'bold', color: cores.texto },
  subtitulo: { color: cores.cinza, marginBottom: 12 },
  input: {
    height: 50,
    paddingHorizontal: 16,
    borderRadius: 10,
    borderWidth: 1,
    borderColor: '#dfe6e9',
    backgroundColor: cores.branco,
  },
  linha: {
    flexDirection: 'row',      // campos lado a lado
    gap: 12,                   // espaço entre eles
  },
  metade: { flex: 1 },         // cada campo ocupa metade da linha
  botao: {
    height: 50,
    marginTop: 8,
    borderRadius: 10,
    backgroundColor: cores.primaria,
    justifyContent: 'center',
    alignItems: 'center',
  },
  botaoTexto: { color: cores.branco, fontWeight: 'bold', fontSize: 16 },
  rodape: { flexDirection: 'row', justifyContent: 'center' },
  link: { color: cores.primaria, fontWeight: 'bold' },
});
```

> **Por que `flexGrow: 1` no `contentContainerStyle`?** Sem ele, o conteúdo da ScrollView tem só a altura dos filhos, e o `justifyContent: 'center'` não tem espaço para centralizar.

5.4 Tela Principal com menu de navegação

Análise do layout: a tela é uma coluna com três blocos: cabeçalho fixo (linha com saudação e avatar), conteúdo rolável (`flex: 1`, ocupa o espaço que sobra) e menu inferior fixo (linha com quatro itens distribuídos por `justifyContent: 'space-around'`). O conteúdo tem cards de resumo lado a lado e uma grade de atalhos com `flexWrap`.

```
┌──────────────────────────┐
│ Olá, Maria!        (MS)  │ ← cabeçalho: row + space-between
├──────────────────────────┤
│ [ Pedidos ] [ Saldo  ]   │ ← cards: row + flex 1
│ Atalhos                  │
│ [  ] [  ] [  ]           │ ← grade: row + wrap + gap
│ [  ] [  ] [  ]           │   (ScrollView com flex 1)
├──────────────────────────┤
│ Início Busca Pedidos Perfil│ ← menu: row + space-around
└──────────────────────────┘
```

Código Fonte (`src/screens/HomeScreen.js`):

```
import { useState } from 'react';
import { View, Text, ScrollView, TouchableOpacity, StyleSheet } from 'react-native';
import { cores } from '../styles/theme';

// Dados dos atalhos (em um app real viriam da API)
const atalhos = ['Pagar', 'Transferir', 'Extrato', 'Cartões', 'Recarga', 'Ajuda'];
// Itens do menu inferior
const menu = ['Início', 'Busca', 'Pedidos', 'Perfil'];
```

```
export default function HomeScreen({ navigation }) {
  // Guarda qual item do menu está selecionado
  const [ativo, setAtivo] = useState('Início');

  return (
    <View style={styles.tela}>
      {/* BLOCO 1: cabeçalho fixo */}
      <View style={styles.cabecalho}>
        <View>
          <Text style={styles.ola}>Olá, Maria!</Text>
          <Text style={styles.subtitulo}>Bom dia</Text>
        </View>
        {/* Avatar: toque para sair e voltar ao login */}
        <TouchableOpacity style={styles.avatar} onPress={() => navigation.replace('Login')}>
          <Text style={styles.avatarTexto}>MS</Text>
        </TouchableOpacity>
      </View>

      {/* BLOCO 2: conteúdo rolável que ocupa o espaço restante */}
      <ScrollView style={{ flex: 1 }} contentContainerStyle={styles.conteudo}>
        {/* Cards de resumo lado a lado */}
        <View style={styles.linhaCards}>
          <View style={[styles.card, { backgroundColor: cores.primaria }]}>
            <Text style={styles.cardValor}>12</Text>
            <Text style={styles.cardRotulo}>Pedidos</Text>
          </View>
          <View style={[styles.card, { backgroundColor: '#00b894' }]}>
            <Text style={styles.cardValor}>R$ 350</Text>
            <Text style={styles.cardRotulo}>Saldo</Text>
          </View>
        </View>

        <Text style={styles.secao}>Atalhos</Text>

        {/* Grade de atalhos: 3 por linha com flexWrap */}
        <View style={styles.grade}>
          {atalhos.map((item) => (
            <TouchableOpacity key={item} style={styles.atalho}>
              <View style={styles.icone} />
              <Text>{item}</Text>
            </TouchableOpacity>
          ))}
        </View>
      </ScrollView>

      {/* BLOCO 3: menu inferior fixo */}
      <View style={styles.menu}>
        {menu.map((item) => (
          <TouchableOpacity key={item} style={styles.menuItem} onPress={() => setAtivo(item)}>
            {/* Bolinha que indica o item ativo */}
            <View style={[styles.menuIcone, ativo === item && styles.menuIconeAtivo]} />
            <Text style={[styles.menuTexto, ativo === item && styles.menuTextoAtivo]}>
              {item}
            </Text>
          </TouchableOpacity>
        ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  tela: { flex: 1, backgroundColor: cores.fundo },
  cabecalho: {
    flexDirection: 'row',            // saudação e avatar na mesma linha
    justifyContent: 'space-between', // um em cada ponta
    alignItems: 'center',            // centralizados na vertical
    paddingHorizontal: 20,
    paddingTop: 60,
    paddingBottom: 16,
```

```
    backgroundColor: cores.branco,
  },
  ola: { fontSize: 22, fontWeight: 'bold', color: cores.texto },
  subtitulo: { color: cores.cinza },
  avatar: {
    width: 44,
    height: 44,
    borderRadius: 22,
    backgroundColor: cores.primaria,
    justifyContent: 'center',
    alignItems: 'center',
  },
  avatarTexto: { color: cores.branco, fontWeight: 'bold' },
  conteudo: { padding: 20, gap: 16 },
  linhaCards: { flexDirection: 'row', gap: 12 },
  card: {
    flex: 1,                // os dois cards dividem a largura igualmente
    padding: 16,
    borderRadius: 12,
  },
  cardValor: { color: cores.branco, fontSize: 22, fontWeight: 'bold' },
  cardRotulo: { color: cores.branco },
  secao: { fontSize: 18, fontWeight: 'bold', color: cores.texto },
  grade: {
    flexDirection: 'row',
    flexWrap: 'wrap',       // quebra para a próxima linha
    justifyContent: 'space-between',
    rowGap: 12,
  },
  atalho: {
    width: '31%',           // 3 por linha (sobra espaço para o space-between)
    aspectRatio: 1,         // altura igual à largura: um quadrado
    borderRadius: 12,
    backgroundColor: cores.branco,
    justifyContent: 'center',
    alignItems: 'center',
    gap: 8,
  },
  icone: { width: 32, height: 32, borderRadius: 8, backgroundColor: '#dfe6e9' },
  menu: {
    flexDirection: 'row',          // itens lado a lado
    justifyContent: 'space-around', // espaço igual ao redor de cada item
    paddingTop: 10,
    paddingBottom: 24,             // espaço para a barra de gestos do celular
    borderTopWidth: 1,
    borderTopColor: '#dfe6e9',
    backgroundColor: cores.branco,
  },
  menuItem: { alignItems: 'center', gap: 4 },
  menuIcone: { width: 22, height: 22, borderRadius: 11, backgroundColor: cores.cinza },
  menuIconeAtivo: { backgroundColor: cores.primaria },
  menuTexto: { fontSize: 12, color: cores.cinza },
  menuTextoAtivo: { color: cores.primaria, fontWeight: 'bold' },
});
```

Este menu é construído "à mão" para praticar Flexbox: ele só muda o item destacado. Na Etapa 6 ele é substituído por um navegador de abas real, que troca de tela de verdade.

Mapa das propriedades usadas no estudo de caso:

Necessidade de layout	Propriedades
Centralizar tudo na tela	`flex: 1` + `justifyContent: 'center'`
Logo/ícone com letra no meio	`justifyContent` + `alignItems: 'center'`
Inputs com largura total	`alignItems: 'stretch'` (padrão)
Um item fora do alinhamento	`alignSelf: 'flex-end'`
Elementos lado a lado	`flexDirection: 'row'`

Necessidade de layout	Propriedades
Dividir a largura igualmente	`flex: 1` em cada filho
Itens nas pontas	`justifyContent: 'space-between'`
Grade que quebra linha	`flexWrap: 'wrap'` + `rowGap`
Conteúdo entre cabeçalho e menu fixos	`flex: 1` no ScrollView

Etapa 6: Navegação entre telas com React Navigation

O React Native não tem rotas por URL como o React web. A navegação é feita pela biblioteca **React Navigation**, que empilha telas (Stack) ou alterna entre abas (Tabs). Nesta etapa ligamos as telas da Etapa 5: Login → Registro, Login → Principal, e a Principal ganha um menu de abas real.

Conceitos:

- **NavigationContainer:** componente que envolve o app inteiro e guarda o estado da navegação (qual tela está aberta). Existe apenas um.
- **Stack Navigator (pilha):** cada nova tela é colocada **em cima** da anterior, como uma pilha de cartas. O botão "voltar" remove a tela do topo.
- **Bottom Tab Navigator (abas):** barra inferior com abas; trocar de aba não empilha telas.
- **Navegadores aninhados:** um navegador pode ser uma tela de outro. Aqui, a tela "Principal" da pilha é um navegador de abas.
- **Props automáticas:** toda tela registrada recebe `navigation` (para navegar) e `route` (para ler parâmetros).

6.1 Instalação

```
# Núcleo da biblioteca
npm install @react-navigation/native

# Dependências nativas: npx expo install escolhe versões compatíveis com o SDK do Expo
npx expo install react-native-screens react-native-safe-area-context

# Navegadores usados neste tutorial
npm install @react-navigation/native-stack @react-navigation/bottom-tabs
```

Os ícones vêm do pacote `@expo/vector-icons`, que já acompanha projetos Expo.

6.2 Métodos de navegação

Método	O que faz	Exemplo de uso
`navigate('Tela', params)`	Vai para a tela (se já estiver na pilha, volta até ela)	Login → Registro
`push('Tela', params)`	Empilha a tela mesmo que ela já exista	Detalhe de produto → outro produto
`goBack()`	Volta para a tela anterior	Botão "Já tenho conta"
`replace('Tela')`	Troca a tela atual (não dá para voltar a ela)	Após o login bem-sucedido
`reset({...})`	Redefine a pilha inteira	Logout

6.3 Arquitetura da navegação

```
NavigationContainer
└── Stack Navigator
    ├── Login        (LoginScreen, sem cabeçalho)
    ├── Registro     (RegisterScreen, com botão voltar)
    └── Principal    (Tab Navigator, sem cabeçalho)
        ├── Início   (HomeScreen)
        ├── Busca    (BuscaScreen)
        ├── Pedidos  (PedidosScreen)
        └── Perfil   (PerfilScreen)
```

6.4 Navegador de abas

Código Fonte (`src/navigation/TabRoutes.js`):

```
import { View, Text } from 'react-native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
import HomeScreen from '../screens/HomeScreen';
import PerfilScreen from '../screens/PerfilScreen';
import { cores } from '../styles/theme';

// Cria o navegador de abas
const Tab = createBottomTabNavigator();

// Tela simples reaproveitada para as abas ainda não implementadas
function EmConstrucao({ route }) {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      {/* route.name = nome da aba atual */}
      <Text>Tela {route.name} em construção</Text>
    </View>
  );
}

// Mapa: nome da aba -> nome do ícone (Ionicons)
const icones = {
  'Início': 'home',
  Busca: 'search',
  Pedidos: 'receipt',
  Perfil: 'person',
};

export default function TabRoutes() {
  return (
    <Tab.Navigator
      // screenOptions vale para todas as abas; recebe a rota atual
      screenOptions={({ route }) => ({
        headerShown: false,                   // a HomeScreen já tem cabeçalho próprio
        tabBarActiveTintColor: cores.primaria, // cor da aba selecionada
        tabBarInactiveTintColor: cores.cinza,
        // Função que desenha o ícone; focused indica se a aba está ativa
        tabBarIcon: ({ focused, color, size }) => (
          <Ionicons
            name={focused ? icones[route.name] : `${icones[route.name]}-outline`}
            size={size}
            color={color}
          />
        ),
      })}
    >
      <Tab.Screen name="Início" component={HomeScreen} />
      <Tab.Screen name="Busca" component={EmConstrucao} />
      <Tab.Screen name="Pedidos" component={EmConstrucao} />
      <Tab.Screen name="Perfil" component={PerfilScreen} />
    </Tab.Navigator>
  );
}
```

> **Ajuste na HomeScreen:** como a barra de abas agora é real, remova o **BLOCO 3** (menu inferior) do `HomeScreen.js` da Etapa 5, junto com o estado `ativo` e os estilos `menu*`.

6.5 Navegador de pilha (rotas principais)

Código Fonte (`src/navigation/AppRoutes.js`):

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import LoginScreen from '../screens/LoginScreen';
import RegisterScreen from '../screens/RegisterScreen';
import TabRoutes from './TabRoutes';
import { cores } from '../styles/theme';

// Cria o navegador de pilha
```

```
const Stack = createNativeStackNavigator();

export default function AppRoutes() {
  return (
    // initialRouteName: primeira tela exibida ao abrir o app
    <Stack.Navigator
      initialRouteName="Login"
      screenOptions={{
        headerStyle: { backgroundColor: cores.primaria }, // fundo do cabeçalho
        headerTintColor: '#fff',                          // cor do título e da seta voltar
      }}
    >
      {/* Login sem cabeçalho: o layout já tem logo e título */}
      <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown: false }} />

      {/* Registro com cabeçalho e seta de voltar automática */}
      <Stack.Screen name="Registro" component={RegisterScreen} options={{ title: 'Cadastro' }} />

      {/* Principal é o navegador de abas (navegação aninhada) */}
      <Stack.Screen name="Principal" component={TabRoutes} options={{ headerShown: false }} />
    </Stack.Navigator>
  );
}
```

6.6 App.js: o ponto de entrada

Código Fonte (`App.js`):

```
import { NavigationContainer } from '@react-navigation/native';
import { StatusBar } from 'expo-status-bar';
import AppRoutes from './src/navigation/AppRoutes';

export default function App() {
  return (
    // NavigationContainer envolve TODA a navegação do app (apenas um por app)
    <NavigationContainer>
      <AppRoutes />
      <StatusBar style="auto" />
    </NavigationContainer>
  );
}
```

As telas da Etapa 5 já funcionam sem alteração: `navigation.navigate('Registro')` abre o cadastro, `navigation.goBack()` volta ao login e `navigation.replace('Principal')` entra no app sem permitir voltar ao login.

6.7 Passando parâmetros e fazendo logout

Conceito: o segundo argumento de `navigate` é um objeto de parâmetros, lido na tela de destino por `route.params` . Para o logout, `reset` limpa a pilha, evitando que o botão voltar retorne à área logada.

Enviando parâmetros (trecho do `LoginScreen.js`):

```
function handleLogin() {
  // Em um app real, o nome viria da resposta da API
  // Parâmetros para um navegador aninhado: { screen: aba, params: dados }
  navigation.replace('Principal', {
    screen: 'Perfil',
    params: { nome: 'Maria Silva', email },
  });
}
```

Recebendo parâmetros e fazendo logout (`src/screens/PerfilScreen.js`):

```
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { cores } from '../styles/theme';

export default function PerfilScreen({ navigation, route }) {
```

```
  // ?. e ?? evitam erro quando a tela abre sem parâmetros
  const nome = route.params?.nome ?? 'Visitante';
  const email = route.params?.email ?? '-';

  function sair() {
    // getParent(): sobe das abas para a pilha principal
    // reset: a pilha passa a conter somente a tela de Login
    navigation.getParent().reset({
      index: 0,
      routes: [{ name: 'Login' }],
    });
  }

  return (
    <View style={styles.container}>
      <Text style={styles.nome}>{nome}</Text>
      <Text style={styles.email}>{email}</Text>

      <TouchableOpacity style={styles.botao} onPress={sair}>
        <Text style={styles.botaoTexto}>SAIR</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', gap: 8 },
  nome: { fontSize: 22, fontWeight: 'bold' },
  email: { color: cores.cinza },
  botao: {
    marginTop: 24,
    paddingHorizontal: 40,
    paddingVertical: 14,
    borderRadius: 10,
    backgroundColor: '#d63031',
  },
  botaoTexto: { color: '#fff', fontWeight: 'bold' },
});
```

Estrutura final do projeto

```
meu-app/
├── App.js                    (NavigationContainer)
└── src/
    ├── navigation/
    │   ├── AppRoutes.js      (Stack: Login, Registro, Principal)
    │   └── TabRoutes.js      (Tabs: Início, Busca, Pedidos, Perfil)
    ├── screens/
    │   ├── LoginScreen.js
    │   ├── RegisterScreen.js
    │   ├── HomeScreen.js     (sem o menu manual)
    │   └── PerfilScreen.js
    └── styles/
        └── theme.js
```

Como testar

1. **Iniciar:** execute `npx expo start` e abra no Expo Go ou no emulador.
2. **Registro:** na tela de login, toque em "Cadastre-se"; a tela de cadastro abre com a seta de voltar no cabeçalho.
3. **Voltar:** toque em "Entrar" ou na seta; o app retorna ao login.
4. **Login:** toque em "ENTRAR"; o app abre a área principal na aba Perfil, exibindo o nome recebido por parâmetro.
5. **Abas:** alterne entre Início, Busca, Pedidos e Perfil; o ícone ativo muda de cor e de formato.
6. **Logout:** em Perfil, toque em "SAIR"; o botão voltar do Android não deve retornar à área logada.

Próximo passo sugerido: substituir os `alert` e o `navigation.replace` por chamadas reais à API NestJS (`POST /users` e `POST /auth/login`), guardando o `access_token` retornado.